
Státnice: rychlá příprava na trojku

Informatika - Umělá inteligence

zaměření **Strojové učení**

Lean studijní text optimalizovaný na **nejméně času k jistému průchodu** (známka 3, >98 %).
Strukturováno přesně podle oficiálních požadavků (požadavky/detailni-pozadavky.pdf).

Jak tento text funguje

- Každé téma má dvě části: šedý box **TÉMA** = **oficiální požadavek doslova** (co se může zkoušet), a pod ním *Učivo* = naše stručné, intuitivní vysvětlení.
- **Odpověď podle bodů** ukazuje, co stačí říct na 1 / 2 / 3 body. **Cíl pro průchod je zelená = 2 body.**
- **Vyzkoušej se** = otázky na aktivní vybavování; **Pozor** = častá past.
- Obrázky jsou tam, kde ušetří čas. Text je **přepsaný pro pochopení**, ne opsaný z poznámek.

Obsah

1	Strategie: jak projít s minimem času	2
1.1	Rubrika zvládnutí (sebehodnocení)	2
2	Rozsah: co se učit a co ne	2
3	Část III: Specializace - Základy umělé inteligence	3
3.1	Řešení úloh prohledáváním	3
3.2	Splňování omezujících podmínek (CSP)	4
3.3	Logické uvažování	5
3.4	Pravděpodobnostní uvažování	6
3.5	Reprezentace znalostí	7
3.6	Automatické plánování	8
3.7	Markovské rozhodovací procesy (MDP)	9
3.8	Hry a teorie her	10
3.9	Strojové učení (přehledově)	11
4	Část IV: Specializace - Strojové učení	13
4.1	Učení s učitelem (základní pojmy, evaluace, regularizace)	13
4.2	Učení založené na příkladech - k nejbližších sousedů (kNN)	14
4.3	Lineární regrese	14
4.4	Logistická regrese	15
4.5	Rozhodovací stromy	16
4.6	Metoda podpůrných vektorů (SVM)	16
4.7	Kombinace více modelů (ensemble)	17
4.8	Statistické testy	18
4.9	Učení bez učitele (shlukování, PCA)	18
	Společná matematika a informatika (připravuje se)	20

1 Strategie: jak projít s minimem času

Bodový rozpočet (proč to stačí)

8 otázek po 0-3 bodech. **Průchod (známka 3)** = aspoň 12 bodů celkem **a** aspoň 5 ze společných okruhů (4 otázky) **a** aspoň 7 ze specializace (4 otázky).

- **Specializace je klíčová** (potřebuješ 7/12, tj. průměr 1,75 na otázku). Sem dej většinu času (ML + Základy UI $\approx 60\%$). Společné okruhy mají nízký floor 5/12 ($\approx 1,25$ na otázku) - stačí breadth, ne hloubka.
- **Hlavní riziko je nula z přidělené otázky**, ne chybějící důkaz. Proto měj u *každého* tématu aspoň „kostku na 1 bod“. Žádné nuly $\Rightarrow$ pravděpodobnost průchodu jde nad 98 %.
- **Trénuj nad hranici** (cíl $\sim 16/24$), protože $5+7=12$ nemá rezervu.
- **Vynech** (bezpečně): dlouhé důkazy, plná odvození, jazykové varianty mimo zvolený jazyk, vše mimo rozsah. Detaily viz sekce 2.

1.1 Rubrika zvládnutí (sebehodnocení)

stav	co umíš
• červená (0)	ani definici
• žlutá (1)	definice + pár pojmů
• zelená (2)	definice + algoritmus/věta + příklad + intuice (cíl)
• modrá (3)	zelená + přesné podmínky/detail (jen u nejdůležitějších)

Cíl: každé téma aspoň žlutě; specializace $\sim 85\%$ zeleně; nejdůležitější modely modře. Podrobná strategie a rozpočet času: **STRATEGY.md** v repozitáři.

2 Rozsah: co se učit a co ne

Závazný zdroj: [pozadavky/detailni-pozadavky.pdf](#). Pro zaměření **Strojové učení** se zkouší: společná matematika (6 témat), společná informatika (4 témata), a ze specializace *Základy UI* (téma 1) a *Strojové učení* (téma 3).

Mezera v poznámkách - ověř / doplň

Záměrně NEzahrnuto (nezkouší se, neuč se): Robotika a NLP; neuronové sítě a hluboké učení (perceptron, MLP, CNN, word2vec patří k NLP); počítačové sítě, databáze a SQL, grafika, softwarové inženýrství, RSA/FFT a pokročilé algoritmy, výpočetní geometrie.

3 Část III: Specializace - Základy umělé inteligence

3.1 Řešení úloh prohledáváním

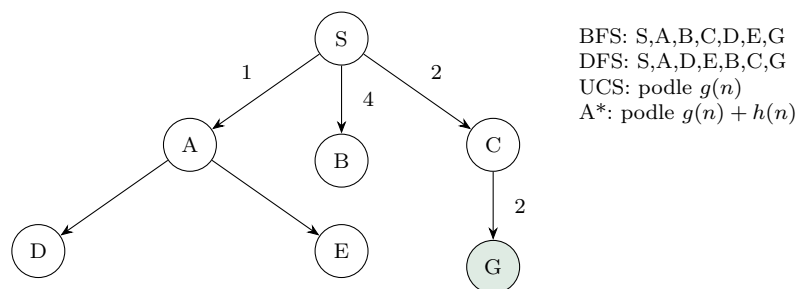
priorita: vysoká cíl: zelená (2 body)

Řešení úloh prohledáváním

- formulace úlohy
- stromové vs. grafové prohledávání
- neinformované prohledávání (DFS, BFS, uniform-cost search)
- informované prohledávání (algoritmus A*, přípustné a konzistentní heuristiky)

Učivo (jak na to):

Prohledávání je vhodné, když stav umíme brát jako uzel grafu a akci jako hranu. Odpověď u zkoušky má vždy říct: jak problém formuluji, jak držím frontier, podle čeho vybírám další uzel a kdy je řešení optimální.



Úloha je dána počátečním stavem, množinou akcí nebo přechodovým modelem, testem cíle a cenou cesty. Stromové prohledávání si pamatuje cesty ve stromu možností, grafové prohledávání navíc uzavírá již navštívené stavy, aby se netočilo v cyklech.

- **BFS:** fronta, bere nejmenší uzel. Je úplné pro konečné větvení a optimální při stejných cenách kroků.
- **DFS:** zásobník nebo rekurze, jde do hloubky. Má malou paměť, ale není obecně optimální a ve stromové verzi se může zacyklit.
- **Uniform-cost search:** prioritní fronta podle dosavadní ceny $g(n)$. Je Dijkstra nad stavovým prostorem, optimální pro nezáporné ceny.
- **A*:** vybírá minimum $f(n) = g(n) + h(n)$. Přípustná heuristika nikdy nepřeceňuje cenu do cíle. Konzistentní heuristika splňuje $h(n) \leq c(n, a, n') + h(n')$ a dává optimalitu i v grafovém prohledávání bez znovutevívání uzlů.

► **Klíč:** BFS = hloubka, DFS = paměť, UCS = skutečná cena, A* = skutečná cena plus odhad.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** definuj stav, akce, počáteční stav, test cíle a cenu. Vysvětli rozdíl tree search vs. graph search a pojmenuj DFS, BFS, UCS, A*.
- 2 + k tomu: u každého algoritmu řekni datovou strukturu frontier a optimalnost: BFS pro jednotkové ceny, UCS pro nezáporné ceny, A* pro přípustnou heuristiku ve stromu a

konzistentní heuristiku v grafu.

3 Bonus: složitosti BFS $O(b^{d+1})$ čas i paměť, DFS čas $O(b^m)$ a paměť $O(bm)$ nebo $O(m)$ při backtrackingu. Dominance heuristiky: větší přípustná heuristika typicky expanduje méně uzlů.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jakých pět částí má formulace problému prohledáváním?
- Kdy je BFS optimální a kdy nestačí?
- Jaký je rozdíl mezi přípustnou a konzistentní heuristikou?
- Co se stane, když DFS pustíš jako tree search na graf s cyklem?

Pozor (častá past): Neříkej, že A^* je vždy optimální. Potřebuje správnou heuristiku a u grafového prohledávání se typicky požaduje konzistence.

3.2 Splňování omezujících podmínek (CSP)

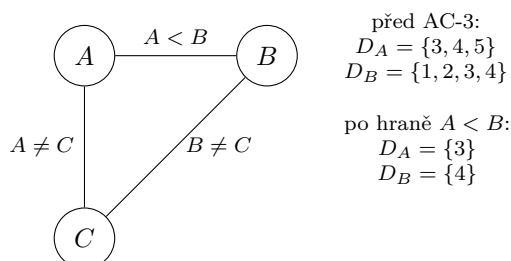
priorita: vysoká cíl: zelená (2 body)

Splňování omezujících podmínek

- problém splňování podmínek
- hranová konzistence (algoritmus AC-3)
- algoritmus hledání řešení (MAC)

Učivo (jak na to):

CSP neřeší cestu akcí, ale tabulku hodnot: každé proměnné chceme dát hodnotu tak, aby všechny podmínky seděly. Velký zisk je škrtnat nemožné hodnoty dřív, než rozvětvíme backtracking.



CSP má proměnné X_i , domény D_i a podmínky jako relace na proměnných. Řešení je úplné přiřazení hodnot, které splní všechny podmínky. Hrana (X_i, X_j) je hranově konzistentní, když pro každou hodnotu $x \in D_i$ existuje hodnota $y \in D_j$, která splní binární podmínku mezi X_i a X_j .

AC-3: vlož všechny orientované hrany do fronty. Vyber (X_i, X_j) a smaž z D_i hodnoty bez podpory v D_j . Pokud se D_i změnila, vrať do fronty všechny hrany (X_k, X_i) kromě právě opačné. Prázdná doména znamená neúspěch. Jinak dostaneš lokálně vyčištěný CSP, ne nutně hotové řešení.

MAC: backtracking nad proměnnými, po každém přiřazení se znovu udržuje hranová konzistence. Tím se často odhalí spor dřív než hluboko ve stromu.

Odpověď podle bodů (cíľ pro průchod: zelená = 2 body)

1 Minimum: CSP = proměnné, domény, podmínky, úplné konzistentní přiřazení. AC-3 škrtná hodnoty bez podpory u sousedních proměnných.

- ② + k tomu: popiš frontu orientovaných hran v AC-3 a MAC jako backtracking plus opakované spuštění AC po každém přiřazení. Přidej mini příklad $A < B$.
- ③ Bonus: AC-3 pro binární podmínky má typicky složitost $O(cd^3)$, kde c je počet podmínek a d velikost domény. Hranová konzistence je jen lokální, proto sama negarantuje řešení.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co přesně znamená, že hodnota má podporu v sousední doméně?
- Proč je hrana v CSP orientovaná, i když podmínka mezi proměnnými je binární?
- Jak se liší forward checking, AC-3 a MAC?
- Proč může být CSP hranově konzistentní a přesto nemít řešení?

Pozor (častá past): Nezaměň hranovou konzistenci s globální konzistencí. AC-3 škrte zjevně nemožné hodnoty, ale nemusí najít celé přiřazení.

3.3 Logické uvažování

priorita: vysoká cíl: zelená (2 body)

Logické uvažování

- základy výrokové logiky (konjunktivní a disjunktivní normální forma)
- algoritmus DPLL
- dopředné a zpětné řetězení (Hornovské klauzule)
- rezoluce

Učivo (jak na to):

Logické uvažování převádí znalosti na formule a otázku na test důsledku. Prakticky chceš umět dvě cesty: SAT přes DPLL a odvozování přes Hornovy klauzule nebo rezoluci.

CNF je konjunkce klauzulí, kde klauzule je disjunkce literálů. DNF je disjunkce konjunkcí literálů. Hornova klauzule má nejvýše jeden pozitivní literál, například $\neg p \vee \neg q \vee r$, což odpovídá pravidlu $p \wedge q \Rightarrow r$.

DPLL rozhoduje splnitelnost CNF. Opakuj jednotkovou propagaci, nastav čisté literály tak, aby splnily své klauzule, zkontroluj konec: žádné klauzule znamenají SAT, prázdná klauzule znamená UNSAT. Když není rozhodnuto, vyber proměnnou a větvi na pravda/nepravda.

Řetězení: dopředné řetězení jde od faktů k novým faktům, je data-driven. Zpětné řetězení jde od cíle k podcílům, je goal-driven. Obojí je levné a přirozené pro Hornovy klauzule.

Rezoluce: pro dotaz α testuj nesplnitelnost $KB \wedge \neg \alpha$ v CNF. Z klauzulí $(A \vee p)$ a $(B \vee \neg p)$ odvoď $(A \vee B)$. Když vznikne prázdná klauzule, $KB \models \alpha$.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- ① **Minimum:** vysvětli CNF, DNF, klauzuli, literál, Hornovu klauzuli a že důsledek lze testovat přes $KB \wedge \neg \alpha$.
- ② + k tomu: popiš kroky DPLL: jednotková propagace, čistý literál, koncové stavy, větvení. Uveď rozdíl mezi dopředným a zpětným řetězením.
- ③ **Bonus:** napiš rezoluční pravidlo a vysvětli rezoluční zamítnutí. DPLL je v nejhorším exponenciální, ale jednotková propagace drasticky zmenšuje praxi.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co udělá DPLL s jednotkovou klauzulí?
- Jak přepíšeš $\neg p \vee \neg q \vee r$ jako pravidlo?
- Proč se při důkazu dotazu přidává $\neg \alpha$?
- Kdy je výhodnější zpětné řetězení než dopředné?

Pozor (častá past): Neplet DPLL a rezoluci: DPLL hledá ohodnocení CNF, rezoluce odvozuje nové klauzule a hledá prázdnou klauzuli.

3.4 Pravděpodobnostní uvažování

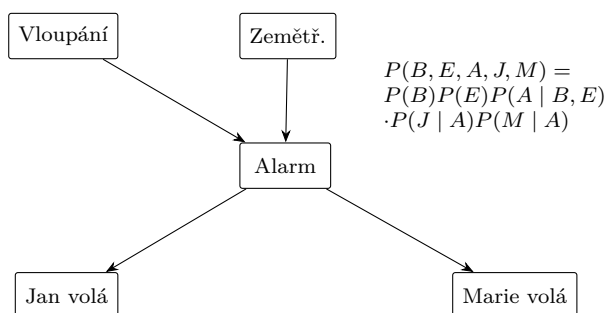
priorita: vysoká cíl: zelená (2 body)

Pravděpodobnostní uvažování

- základy teorie pravděpodobnosti (úplná sdružená distribuce, nezávislost, Bayesovo pravidlo)
- pravděpodobnostní uvažování (vysčítání, normalizace)
- Bayesovské sítě
- konstrukce a vztah k úplné sdružené distribuci
- exaktní odvozování (enumerace, eliminace proměnných)
- aproximační odvozování (Monte Carlo, zamítání, vážení věrohodností)

Učivo (jak na to):

Pravděpodobnost je způsob, jak uvažovat při nejistotě. Úplná sdružená distribuce umí odpovědět na všechno, ale je obrovská. Bayesovská síť ji komprimuje pomocí podmíněných nezávislostí.



Bayesovská síť je orientovaný acyklický graf náhodných proměnných a tabulky $P(X_i | Parents(X_i))$. Společná distribuce se faktorizuje jako

$$P(x_1, \dots, x_n) = \prod_i P(x_i | parents(x_i)).$$

Základní počty: z úplné sdružené distribuce získáš marginál vysčítáním skrytých proměnných, například $P(X) = \sum_y P(X, y)$. Podmíněnou pravděpodobnost získáš normalizací: $P(X | e) = \alpha P(X, e)$. Bayesovo pravidlo je $P(H | e) = \alpha P(e | H)P(H)$.

Inference v síti: enumerace přímo počítá přes skryté proměnné. Eliminace proměnných průběžně tvoří faktory a počítá jen jednou, takže šetří opakovanou práci. Aproximace generuje vzorky: Monte Carlo obecně, zamítání zahodí vzorky neslučitelné s evidencí, vážení věrohodností vzorky nezahazuje, ale váží je pravděpodobnostmi evidence.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** řekni úplná sdružená distribuce, nezávislost, Bayesovo pravidlo, vysčítání a normalizace. Bayesovská síť = DAG plus podmíněné tabulky.
- 2 + k tomu: napiš faktorizaci sítě a ukaž ji na malé síti. Popiš enumeraci, eliminaci proměnných a rozdíl mezi zamítáním a vážením věrohodností.
- 3 **Bonus:** vysvětli, že konstrukce sítě závisí na pořadí proměnných a nejlepší je obvykle směr od příčin k důsledkům. Eliminace závisí na pořadí eliminovaných proměnných.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak dostaneš $P(X | e)$ z $P(X, e)$?
- Co přesně faktorizuje Bayesovská síť?
- Proč je eliminace proměnných úspornější než čistá enumerace?
- Jaký je rozdíl mezi zamítáním a vážením věrohodností?

Pozor (častá past): Neříkej, že chybějící hrana znamená obyčejnou nezávislost. V Bayesovské síti jde hlavně o podmíněné nezávislosti vzhledem k rodičům a evidenci.

3.5 Reprezentace znalostí

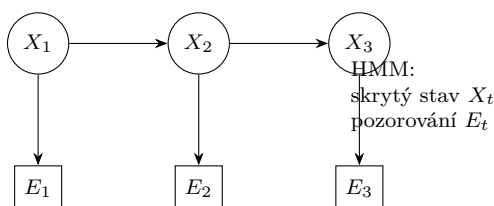
priorita: vysoká cíl: zelená (2 body)

Reprezentace znalostí

- situační kalkulus, problém rámce
- Markovské modely
- filtrace, predikce, vyhlazování, nejpravděpodobnější průchod
- skryté Markovské modely (HMM) vs. dynamické Bayesovské sítě

Učivo (jak na to):

Reprezentace znalostí řeší, jak zapsat svět tak, aby nad ním šlo odvozovat. Situační kalkulus je logický zápis změn po akcích, Markovské modely jsou pravděpodobnostní zápis vývoje stavu v čase.



Situační kalkulus popisuje situace vzniklé posloupností akcí, predikáty o situacích a akci $Result(s, a)$. Problém rámce je otázka, jak stručně zapsat, co se akcí nemění. HMM má skrytý stav X_t , pozorování E_t , přechod $P(X_t | X_{t-1})$ a senzorový model $P(E_t | X_t)$.

Markovské úlohy: filtrace počítá $P(X_t | e_{1:t})$, tedy aktuální stav po dosavadních pozorováních. Predikce počítá budoucí stav $P(X_{t+k} | e_{1:t})$. Vyhlazování zpřesňuje minulý stav $P(X_k | e_{1:t})$ pro $k < t$. Nejpravděpodobnější průchod hledá nejpravděpodobnější posloupnost skrytých stavů, typicky Viterbiho algoritmem.

HMM vs. dynamická Bayesovská síť: HMM má jednu skrytou stavovou proměnnou na čas. Dynamická Bayesovská síť rozkládá stav na více proměnných a dovolí bohatší závislosti mezi nimi.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** situační kalkulus = logika akcí a situací, problém rámce = jak nevyjmenovat všechny nezměněné fakty. HMM = skrytý stav, pozorování, přechodový a senzorový model.
- 2 + k tomu: rozliš filtraci, predikci, vyhlazování a nejpravděpodobnější průchod. Nakresli řetěz $X_1 \rightarrow X_2 \rightarrow X_3$ s pozorováními E_t .
- 3 **Bonus:** uveď, že HMM je speciální případ dynamické Bayesovské sítě; DBN dovolí faktorizovat stav na více proměnných a tím zmenšit tabulky.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je problém rámce v jedné větě?
- Jaký dotaz odpovídá filtraci a jaký vyhlazování?
- Co hledá Viterbiho algoritmus?
- Proč je DBN obecnější než HMM?

Pozor (častá past): Nezaměň filtraci s vyhlazováním: filtrace odhaduje současnost z minulých pozorování, vyhlazování zpětně zpřesňuje minulost i pomocí pozdější evidence.

3.6 Automatické plánování

priorita: vysoká cíl: zelená (2 body)

Automatické plánování

- formulace plánovacího problému (definice operátoru)
- dopředné a zpětné plánování

Učivo (jak na to):

Plánování je prohledávání nad faktorizovaným stavem. Místo černé skříňky používáme fakta, předpoklady akcí a efekty akcí, takže lze rozumněji vybírat relevantní kroky.

Klasický plánovací problém má počáteční stav jako množinu faktů, cíl jako podmínku na fakta a operátory. Operátor má preconditions, tedy kdy ho lze použít, a effects, tedy co přidá nebo smaže ve stavu. Plán je posloupnost operátorů, která z počátečního stavu vyrobí stav splňující cíl.

Mini příklad: operátor $Přesuň(x, A, B)$ má předpoklady $Na(x, A)$ a $Volne(B)$, efekty přidat $Na(x, B)$ a smazat $Na(x, A)$. Dopředné plánování začíná v počátečním stavu a zkouší použitelné akce. Zpětné plánování začíná cílem a hledá akce, které by cíl mohly splnit, přičemž jejich předpoklady se stanou novými podcíli.

- **Dopředné:** jednoduché simulování akcí, ale může zkoušet mnoho irelevantních kroků.
- **Zpětné:** soustředí se na cíl, ale musí řešit, zda vybrané akce jsou vzájemně slučitelné.
- **Kdy použít:** dopředné je přirozené, když je málo použitelných akcí; zpětné, když cíl silně omezuje relevantní akce.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** plánovací problém = počáteční stav, cílová podmínka, operátory. Operátor = předpoklady a efekty. Plán = posloupnost akcí.
- 2 + k tomu: vysvětli dopředné a zpětné plánování na mini příkladu s přesunem objektu.

Řekni, proč faktorizovaný stav umožní pracovat s relevancí akcí.

3 Bonus: uveď rozdíl proti obyčejnému prohledávání: stav není atomický, akce mají logickou strukturu a plánovač může využívat předpoklady a efekty místo slepé expanze.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaké části má operátor?
- Co je stav v klasickém plánování?
- Jak se liší dopředné a zpětné plánování?
- Proč je plánování vhodné pro faktorizované stavy?

Pozor (častá past): Neodpovídej jen obecně "hledání cesty". U plánování musí zaznít preconditions, effects a cíl jako podmínka na fakta.

3.7 Markovské rozhodovací procesy (MDP)

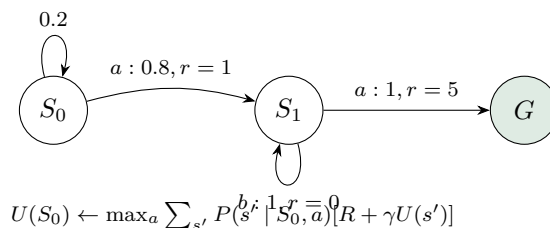
priorita: vysoká cíl: zelená (2 body)

Markovské rozhodovací procesy (MDP)

- formulace problému (výpočet užitku, strategie)
- Bellmanova rovnice
- iterace hodnot, iterace strategií
- POMDP (základní definice)

Učivo (jak na to):

MDP je plánování, kde akce nejsou jisté. Agent volí akci podle stavu, dostane odměnu a pravděpodobnostně přejde do dalšího stavu. Hledáme strategii s nejvyšším očekávaným diskontovaným užitekem.



MDP je čtveřice stavů, akcí, přechodových pravděpodobností $P(s' | s, a)$ a odměn $R(s, a, s')$, často s diskontem γ . Strategie $\pi(s)$ říká, jakou akci zvolit ve stavu s . Užitečnost stavu je očekávaný součet budoucích diskontovaných odměn.

Bellmanova rovnice optimality:

$$U(s) = \max_a \sum_{s'} P(s' | s, a) (R(s, a, s') + \gamma U(s')).$$

Iterace hodnot: opakovaně aktualizuj $U(s)$ Bellmanovou rovnicí a nakonec z ní vyčti strategii.

Iterace strategií: střídá vyhodnocení současné strategie a její zlepšení výběrem lepší akce.

POMDP: stav není přímo pozorovaný, agent udržuje belief, tedy distribuci přes možné stavy.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** MDP = stavy, akce, přechody, odměny, strategie. Cíl je maximalizovat očekávaný diskontovaný užitek.
- 2 + **k tomu:** napiš Bellmanovu rovnici a popiš iteraci hodnot i iteraci strategií. Uveď malý přechod se dvěma možnými následníky.
- 3 **Bonus:** POMDP = MDP s částečnou pozorovatelností, rozhoduje se nad belief stavem. Iterace strategií bývá dražší v kroku vyhodnocení, ale často potřebuje méně iterací.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je strategie v MDP?
- Kde v Bellmanově rovnici vystupuje nejistota?
- Jak se liší iterace hodnot a iterace strategií?
- Co je belief stav u POMDP?

Pozor (častá past): Nepleť MDP s HMM: v MDP agent volí akce a optimalizuje odměny, v HMM hlavně odhaduje skrytý stav z pozorování.

3.8 Hry a teorie her

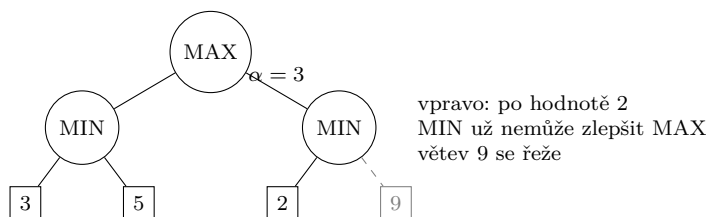
priorita: vysoká cíl: zelená (2 body)

Hry a teorie her

- algoritmy Minimax a alfa-beta prořezávání
- základy teorie her (věžňovo dilemma, Nashovo ekvilibrium)
- mechanism design (typy aukcí)

Učivo (jak na to):

Ve hrách neoptimalizuješ proti přírodě, ale proti jinému agentovi. Minimax předpokládá, že soupeř hraje proti nám co nejlépe. Alfa-beta jen šetří práci, výsledek minimaxu nemění.



Minimax počítá hodnotu hry s nulovým součtem: v uzlu MAX bere maximum z potomků, v uzlu MIN minimum. Nashovo ekvilibrium je profil strategií, kde se žádnému hráči nevyplatí jednostranně změnit strategii.

Alfa-beta: drží α jako nejlepší zatím zajištěnou hodnotu pro MAX a β jako nejlepší zatím zajištěnou hodnotu pro MIN. Když je jasné, že větev nemůže ovlivnit rozhodnutí předka, neprohledává se.

Věžňovo dilemma: každý má individuální motivaci zradit, i když oboustranná spolupráce je společně lepší. Ukazuje rozdíl mezi individuální racionalitou a kolektivním výsledkem.

Aukce: anglická je vzestupná, holandská sestupná, první cena znamená platbu vlastní nabídky, druhá cena typicky motivuje nabídnout skutečnou hodnotu.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** minimax = MAX maximalizuje, MIN minimalizuje. Alfa-beta prořezává větve bez změny výsledku. Nash = žádná jednostranně výhodná odchylka.
- 2 + **k tomu:** ukaž výpočet malého stromu a jeden alfa-beta řez. Vysvětli věžňovo dilema a vyjmenuj základní typy aukcí.
- 3 **Bonus:** dobré pořadí tahů dává alfa-beta velkou úsporu, v ideálním případě zhruba na efektivní hloubku poloviny stromu. U aukce druhé ceny zmiň pravdivé nabízení jako dominantní strategii v jednoduchém modelu.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak se počítá hodnota v MAX a MIN uzlu?
- Proč alfa-beta nemění výsledek minimaxu?
- Co znamená Nashovo ekvilibrium?
- Jaký je rozdíl mezi aukcí první a druhé ceny?

Pozor (častá past): Neříkej, že alfa-beta je jiná strategie hraní. Je to optimalizace výpočtu stejného minimaxového rozhodnutí.

3.9 Strojové učení (přehledově)

priorita: vysoká

cíl: zelená (2 body)

Strojové učení

- základní druhy učení (s učitelem, bez učitele, zpětnovazební)
- rozhodovací stromy (definice, konstrukce)
- regrese, SVM (základní principy)
- Bayesovské učení, EM algoritmus
- zpětnovazební učení
- pasivní učení (definice, metody ADP a TD)
- aktivní učení (definice, explorační vs. exploitační, Q-učení, SARSA)

Učivo (jak na to):

Tady se nečeká detail celé specializace Strojové učení, ale mapa pojmů v kontextu UI. Bezpečná odpověď vždy řekne typ úlohy, co se učí, z jakých dat a jak se pozná dobrý výsledek.

Učení s učitelem má dvojice vstup plus správný výstup a řeší klasifikaci nebo regresi. Učení bez učitele hledá strukturu bez cílových štítků. Zpětnovazební učení má agenta, akce, odměny a učí strategii z interakce s prostředím.

Rozhodovací strom: vnitřní uzel testuje atribut, hrana odpovídá výsledku testu a list dává predikci. Konstrukce vybírá test, který nejlépe rozdělí data, typicky podle informačního zisku nebo Gini nečistoty, a zastavuje nebo prořezává kvůli přeučení.

Regrese a SVM: regrese predikuje spojitou hodnotu, typicky minimalizuje chybu. SVM hledá oddělující nadrovinu s maximálním marginem; pro neseparabilní data používá měkký margin a pro nelineární hranice jádra.

Bayesovské učení a EM: Bayesovské učení kombinuje prior a věrohodnost na posterior $P(h \mid \text{data})$. EM řeší skryté proměnné střídáním E-kroku, který dopočítá očekávání skrytých přiřazení, a M-kroku,

který zlepší parametry.

Zpětnovazební učení: pasivní učení hodnotí pevně danou strategii. ADP odhaduje model přechodů a odměn a pak řeší MDP. TD aktualizuje hodnoty z rozdílu mezi po sobě jdoucími odhady. Aktivní učení současně volí akce: musí vyvažovat explorační a exploitační. Q-učení je off-policy aktualizace hodnot akcí, SARSA je on-policy aktualizace podle skutečně provedeného přechodu (s, a, r, s', a') .

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- ❶ **Minimum:** rozliš učení s učitelem, bez učitele a zpětnovazební. Definuj rozhodovací strom, regresi, SVM a RL slovníkem vstup, výstup, cíl.
- ❷ **+ k tomu:** popiš konstrukci stromu, princip marginu u SVM, Bayesovské učení přes posterior, EM jako E/M kroky, pasivní ADP/TD a aktivní Q-učení/SARSA.
- ❸ **Bonus:** zdůrazni rozdíl ADP vs. TD: ADP se učí model a řeší ho, TD se učí hodnoty přímo. Q-učení je off-policy s maximem přes další akce, SARSA je on-policy a používá další skutečně zvolenou akci.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak rozlišíš klasifikaci a regresi?
- Co je list a co vnitřní uzel v rozhodovacím stromu?
- Co se děje v E-kroku a M-kroku EM algoritmu?
- Jaký je rozdíl mezi Q-učením a SARSA?

Pozor (častá past): Nezajížděj mimo oficiální přehled: žádné hluboké učení, NLP ani robotika. Tady stačí základní principy uvedených metod.

4 Část IV: Specializace - Strojové učení

Toto je **nejdůležitější** okruh: vlastní zaměření a vázaná podmínka (ze specializace potřebuješ aspoň 7/12).
Cíl: každé téma aspoň *zeleně* (2 body), nejdůležitější modely (lineární/logistická regrese, SVM, stromy, k-means, PCA) i *modře*.

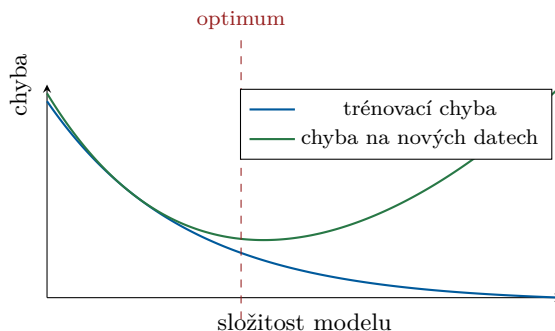
4.1 Učení s učitelem (základní pojmy, evaluace, regularizace)

priorita: vysoká cíl: modrá (3 body) - rámec pro celý okruh

Učení s učitelem: klasifikace a regrese jako základní typy úloh; standardní evaluační metriky; ohodnocení modelu (testovací data, křížová validace, maximální věrohodnost); přeučení a regularizace (generalizační chyba, včasné zastavení trénování, L2 a L1 regularizace); prokletí dimenzionality.

Učivo (jak na to):

Z trénovacích dvojic (vstup, správný výstup) hledáme funkci, která dobře předpoví výstup i pro **nová** data. Nejde o to namemorovat trénink, ale *generalizovat*.



Příliš jednoduchý model = podtrénování; příliš složitý = přeučení. Hledáme dno zelené křivky.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** *klasifikace* = výstup je třída, *regrese* = výstup je číslo. Model trénujeme na trénovacích datech a hodnotíme na *oddělených testovacích* datech.
- 2 + k tomu: *Generalizační chyba* = chyba na nových datech. *Přeučení* = model sedí na šumu trénovacích dat (malá trénovací, velká testovací chyba). Bráníme se: *regularizací* (L2 = penalizuje velké váhy $\lambda \|w\|_2^2$, hladší model; L1 = $\lambda \|w\|_1$, navíc dělá řídké váhy, tj. výběr příznaků), *včasným zastavením* trénování a více daty. *Křížová validace* (k-fold): data rozdělím na k částí, postupně každou použiju jako test, průměruju - spolehlivější odhad chyby než jediné rozdělení.
- 3 **Bonus:** *metriky* - klasifikace: accuracy, precision, recall, F1 (z matice záměn); regrese: MSE/RMSE. *Maximální věrohodnost (MLE)*: parametry volíme tak, aby data byla nejpravděpodobnější; minimalizace záporné log-věrohodnosti je často přesně ta loss funkce. *Prokletí dimenzionality*: v mnoha rozměrech jsou data řídká a vzdálenosti ztrácejí smysl, je třeba exponenciálně víc dat.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak poznám přeučení z trénovací a testovací chyby?
- Jaký je rozdíl mezi L1 a L2 regularizací?

- K čemu je k-fold křížová validace a proč je lepší než jedno rozdělení?
- Co znamená precision vs recall?

Pozor (častá past): Model se nikdy nelaď (ani nevybírám) podle testovacích dat - na to je validační sada. Vysoká accuracy u nevyvážených tříd klame (proto F1).

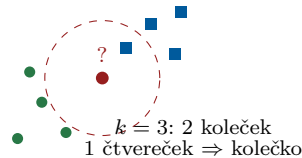
4.2 Učení založené na příkladech - k nejbližších sousedů (kNN)

priorita: střední cíl: zelená (2 body)

Učení založené na příkladech: algoritmus k-nejbližších sousedů.

Učivo (jak na to):

„Řekni mi, kdo jsou tvoji sousedé, a já ti řeknu, kdo jsi.“ Nový bod dostane třídu podle většiny svých k nejbližších trénovacích bodů. Nic se netrénuje - jen se pamatují data.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 Minimum:** Pro nový bod najdi k nejbližších trénovacích bodů (podle vzdálenosti, typicky euklidovské) a přiřaď *většinou* třídu (u regrese průměr).
- 2 + k tomu:** Žádné trénování (líné učení), veškerá práce je při dotazu. Malé k = citlivé na šum (přeučení), velké k = hladší, ale rozmazává hranice. k se volí křížovou validací. Nutné škálovat příznaky.
- 3 Bonus:** Trpí prokletím dimenzionality a je pomalé při mnoha datech (dotaz prochází vše).

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co se stane při $k = 1$ a co při $k = \text{počet všech bodů}$?
- Proč je nutné příznaky normalizovat?

Pozor (častá past): kNN nic „nenatrénuje“ - náročný je až dotaz. A bez škálování dominuje příznak s velkým rozsahem.

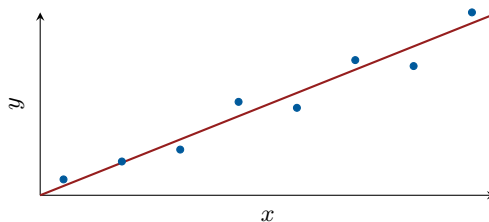
4.3 Lineární regrese

priorita: vysoká cíl: modrá (3 body)

Lineární regrese: analytické řešení metodou nejmenších čtverců; trénování pomocí stochastic gradient descent.

Učivo (jak na to):

Proložíme daty **přímku** (obecně nadrovinu) tak, aby součet *čtverců* svislých odchylek byl co nejmenší.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- ❶ **Minimum:** Model $y = w^\top x + b$. Minimalizujeme $MSE \sum_i (y_i - w^\top x_i)^2$ (metoda nejmenších čtverců).
- ❷ **+ k tomu:** *Analytické řešení* (normální rovnice): $\hat{w} = (X^\top X)^{-1} X^\top y$, kde sloupec jedniček v X pokryje intercept b . Funguje, dokud $X^\top X$ není moc velká / singulární.
- ❸ **Bonus:** *SGD* - místo přesného vzorce iterativně: $w \leftarrow w - \eta \nabla_w \text{loss}$ počítané po malých dávkách dat; nutné u velkých dat. η = rychlost učení (moc velká diverguje, moc malá je pomalá).

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jak zní normální rovnice a kdy ji nelze (rozumně) použít?
- Co řídí parametr η u SGD?

Pozor (častá past): Nejmenší čtverce trestají odlehlé body kvadraticky - jsou citlivé na outliery.

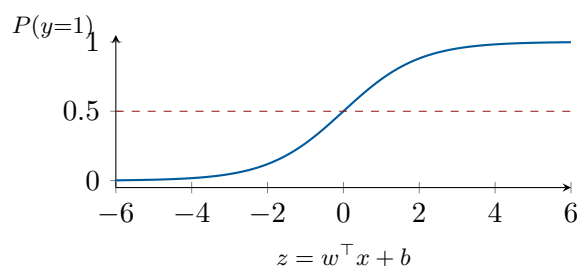
4.4 Logistická regrese

priorita: vysoká cíl: modrá (3 body)

Logistická regrese: binární klasifikace (sigmoid, metody trénování); klasifikace do více tříd (softmax, metody trénování).

Učivo (jak na to):

Navzdory názvu je to **klasifikátor**: lineární skóre $w^\top x$ protlačíme funkcí sigmoid, aby vyšla *pravděpodobnost* třídy mezi 0 a 1.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- ❶ **Minimum:** $P(y=1 | x) = \sigma(w^\top x + b)$, kde $\sigma(z) = \frac{1}{1+e^{-z}}$. Klasifikuji podle prahu 0,5.
- ❷ **+ k tomu:** Trénuje se maximalizací věrohodnosti = minimalizací *cross-entropy* (*log-loss*) gradientním sestupem (analytické řešení neexistuje). Hranice je lineární.
- ❸ **Bonus:** Více tříd přes *softmax* $P(y=k) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ (zobecnění sigmoidu); loss opět cross-entropy.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Proč se nepoužívá MSE, ale cross-entropy?
- Čím se liší sigmoid a softmax?

Pozor (častá past): Logistická regrese je klasifikace, ne regrese. Hranice je lineární (na nelineární potřebuješ příznaky navíc nebo jádro).

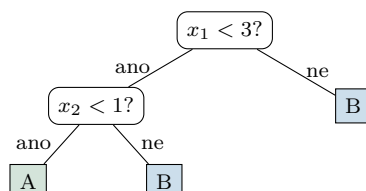
4.5 Rozhodovací stromy

priorita: vysoká cíl: zelená (2 body)

Rozhodovací stromy: algoritmus učení a kritéria větvení; prořezávání.

Učivo (jak na to):

Posloupnost otázek typu „je příznak $>$ práh?“. Každá otázka rozdělí data; chceme otázky, po kterých jsou výsledné skupiny co *nejčistší* (jedna třída).



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** Strom dělí data podmínkami na příznacích; v listech je predikce (třída/hodnota). Staví se hladově shora.
- 2 **+ k tomu:** V každém uzlu vyber dělení, které nejvíc *zvýší čistotu* - kritéria: *informační zisk* (pokles *entropie*) nebo *Gini index*. Rekurzivně až do čistých listů / zastavovacího kritéria.
- 3 **Bonus:** Plně narostlý strom se přeučí $\Rightarrow$ *prořezávání* (pre-pruning: zastav brzy podle hloubky/min. vzorků; post-pruning: ořež větev, které nezlepší chybu na validaci). Výhoda: interpretovatelnost.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co měří entropie / Gini a co je informační zisk?
- Proč a jak se strom prořezává?

Pozor (častá past): Hluboký strom se skoro vždy přeučí. Stromy také snadno preferují příznaky s mnoha hodnotami.

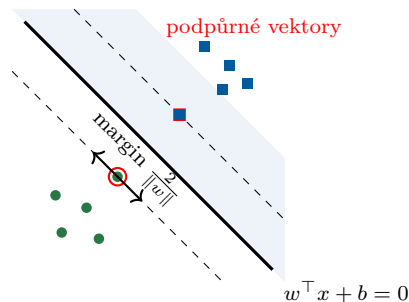
4.6 Metoda podpůrných vektorů (SVM)

priorita: vysoká cíl: zelená (2 body)

Metoda podpůrných vektorů: klasifikátor pro lineárně separabilní třídy; klasifikátor pro lineárně neseperabilní třídy; jádrové funkce; klasifikace do více tříd.

Učivo (jak na to):

Mezi dvě třídy chceme proložit **co nejširší ulici** (pruh bez bodů). Hranice vede středem. Širší ulice = lepší generalizace.



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** Najdi nadrovinu oddělující třídy s *největším marginem*; klasifikace $\text{sign}(w^T x + b)$.
- 2 **+ k tomu:** *Podpůrné vektory* = body na okraji marginu (u měkkého marginu i uvnitř marginu / chybně klasifikované), jen ty určují hranici. Neseparabilní data: *měkký margin* se slack ξ_i a penaltou $C \sum \xi_i$ (C = kompromis úzký margin vs. chyby). *Jádro* $K(x, x')$ nahradí skalární součin a oddělí data ve vyšší dimenzi bez explicitní transformace (RBF, polynom).
- 3 **Bonus:** úloha $\min \frac{1}{2} \|w\|^2$ za $y_i(w^T x_i + b) \geq 1$; více tříd přes one-vs-rest nebo one-vs-one.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co jsou podpůrné vektory a co dělá parametr C ?
- Proč jádro umožní oddělit neseparabilní data?

Pozor (častá past): Velké C není „lepší“ - úzký margin se přeučí. Jádro nemění SVM, jen skalární součin.

4.7 Kombinace více modelů (ensemble)

priorita: vysoká cíl: zelená (2 body)

Kombinace více modelů: bagging a boosting; metoda náhodných lesů.

Učivo (jak na to):

Více slabších modelů dohromady často předčí jeden velký - „moudrost davu“. Liší se tím, *jak* je spojíme.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** *Bagging* trénuje modely *paralelně* na náhodných podvýběrech dat (bootstrap) a *průměruje/hlasuje* - snižuje *rozptyl*. *Boosting* staví modely *sekvenčně*, každý opravuje chyby předchozích - často snižuje *bias*, ale je citlivější na šum.
- 2 **+ k tomu:** *Náhodný les* = bagging rozhodovacích stromů + v každém uzlu se uvažuje jen náhodná podmnožina příznaků (dekoreluje stromy). Robustní, málo ladění.
- 3 **Bonus:** Boosting (např. AdaBoost) zvyšuje váhu špatně klasifikovaných příkladů; silný, ale citlivější na šum/přeučení než bagging.

Vyzkoušej se (zakryj text a odpověz nahlas)

- Bagging vs boosting: paralelně/sekvenčně, snižuje rozptyl/bias?
- Čím se náhodný les liší od prostého baggingu stromů?

Pozor (častá past): Boosting nezaměňuj s baggingem: boosting je sekvenční a typicky cílí hlavně na bias, bagging paralelní na rozptyl.

4.8 Statistické testy

priorita: střední cíl: zelená (2 body)

Statistické testy: studentův t-test (jednovýběrový a dvouvýběrový); chí-kvadrát test (test dobré shody).

Učivo (jak na to):

Ptáme se: je pozorovaný rozdíl **reálný**, nebo jen náhoda? Spočteme testovou statistiku a porovnáme s tím, co by dělala náhoda za platnosti *nulové hypotézy*.

Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** Nulová hypotéza H_0 = „žádný efekt“. Test dá *p-hodnotu*; když $p < \alpha$ (typicky 0,05), H_0 zamítáme.
- 2 **+ k tomu:** *t-test* porovnává *střední hodnoty*: jednovýběrový (průměr vs. daná konstanta), dvouvýběrový (dva průměry proti sobě); předpokládá zhruba normální data. *Chí-kvadrát test dobré shody* porovnává *pozorované vs. očekávané četnosti* kategorií: $\chi^2 = \sum \frac{(O-E)^2}{E}$.
- 3 **Bonus:** chyba I. druhu = zamítnu platné H_0 ($= \alpha$); chyba II. druhu = nezamítnu neplatné H_0 .

Vyzkoušej se (zakryj text a odpověz nahlas)

- Co je p-hodnota a kdy zamítám H_0 ?
- Kdy t-test a kdy chí-kvadrát?

Pozor (častá past): Malá p-hodnota neznamena „velký efekt“, jen „těžko vysvětlitelný náhodou“.

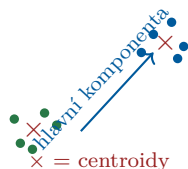
4.9 Učení bez učitele (shlukování, PCA)

priorita: vysoká cíl: zelená (2 body)

Učení bez učitele: shlukování (algoritmus k-means); hierarchické shlukování; redukce dimenzionality (analýza hlavních komponent, PCA).

Učivo (jak na to):

Nemáme správné odpovědi, jen data. Hledáme v nich *strukturu*: přirozené skupiny (shlukování) nebo směry, kde se data nejvíc mění (PCA).



Odpověď podle bodů (cíl pro průchod: zelená = 2 body)

- 1 **Minimum:** *Shlukování* = rozděl data do skupin podle podobnosti. *PCA* = najdi pár směrů, které zachytí nejvíc rozptylu, a do nich data promítni (zmenšení dimenze).
- 2 **+ k tomu:** *k-means*: zvol k center, opakuj {přiřaď body nejbližšímu centru; přepočítej centra jako průměry} až do ustálení. *Hierarchické shlukování*: postupně slučuj nejbližší shluky (aglomerativně) $\Rightarrow$ *dendrogram*, k není třeba dopředu.
- 3 **Bonus:** *PCA* = vlastní vektory kovarianční matice (největší vlastní čísla = nejvíc rozptylu). *k-means*: nutné zvolit k a citlivý na počáteční centra (*k-means++*).

Vyzkoušej se (zakryj text a odpověz nahlas)

- Jaké dva kroky *k-means* opakuje?
- Co maximalizují hlavní komponenty a jak souvisí s kovarianční maticí?
- Výhoda hierarchického shlukování oproti *k-means*?

Pozor (častá past): *k-means* hledá zhruba kulové, podobně velké shluky a vyžaduje předem k ; na protáhlé/různě husté shluky selhává.

Společná matematika a informatika

Připravuje se

Společné okruhy (matematika + informatika) jsou nižší priorita (floor 5/12) a doplní se ve stejném formátu. Do té doby použij oficiální požadavky a poznámky v submodulu **notes-ipp**.